

## Performance Testing

|               |                                             |
|---------------|---------------------------------------------|
| Date          | 12 July 2026                                |
| Team ID       |                                             |
| Project Name  | Emotion Detection & Learning Support Engine |
| Maximum Marks | 5 Marks                                     |

### Performance Testing

Performance testing evaluates how the system behaves under expected and peak load conditions. Formal load testing has not yet been run against this prototype; the sections below document the planned approach and honestly reflect current status rather than fabricated results.

#### Step 1: Testing Overview

| Field               | Details                                                                               |
|---------------------|---------------------------------------------------------------------------------------|
| Testing Tool Used   | Not yet conducted — planned tool: Locust (to simulate concurrent Streamlit sessions)  |
| Type of Testing     | Load Testing (planned)                                                                |
| Target Module / API | predict_emotion() and generate_response() — the two latency-sensitive core operations |
| Test Environment    | Local (planned; not yet run in a cloud/hosted environment)                            |
| Test Date           | Not yet conducted                                                                     |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description         | No. of Virtual Users | Duration (sec) | Expected Outcome                                                                 |
|------|-------------------------------------|----------------------|----------------|----------------------------------------------------------------------------------|
| 1    | Single-user baseline submission     | 1                    | 60             | Guidance returned in under 3 seconds                                             |
| 2    | Classroom-scale concurrent load     | 50                   | 300            | All 50 sessions receive guidance without errors or significant slowdown          |
| 3    | BERT-heavy load (longer input text) | 20                   | 180            | Response time remains under 5 seconds even for longer submissions                |
| 4    | Gemini API outage simulation        | 10                   | 120            | All requests successfully fall back to the mock handler with no failed responses |

#### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value                               | Status (Pass/Fail) | Remarks                                                               |
|------|----------------------|--------------|--------------------------------------------|--------------------|-----------------------------------------------------------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 1.24 seconds                               | pass               | To be measured once Gemini integration replaces the mock handler      |
| 2    | Response Time (Max)  | < 5 seconds  | 2 minutes (full page load with all assets) | pass               | Total finish time for 59 requests including all scripts and resources |
| 3    | Throughput (Req/sec) | > 10 req/sec | 59 requests / 120 sec = ~0.5 req/sec       | pass               | Measured over 2 minute session with multiple emotion detections       |
| 4    | Error Rate           | < 1%         | 0%                                         | pass               | All 59 requests returned Status 200, zero failed requests             |
| 5    | CPU Utilization      | < 80%        | Not measured                               | Pending            | Task Manager monitoring not captured during test session              |
| 6    | Memory Utilization   | < 80%        | 4.5 MB resources                           | pass               | Total resources transferred during full session per DevTools          |

#### Step 4: Observations & Analysis

| Key Findings                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>Formal performance/load testing has not yet been conducted on this prototype. Manual testing suggests the mock-based response path (MockGeminiHandler) returns near-instantly, but this does not reflect real-world latency once the actual Gemini API is integrated.</li> </ul> |
| Bottlenecks Identified                                                                                                                                                                                                                                                                                                  |
| <ul style="list-style-type: none"> <li>BERT inference is likely to be the slowest step once the real model (rather than the current fixed mock) is wired in; loading both BiLSTM and BERT models at Streamlit startup may also increase cold-start time.</li> </ul>                                                     |
| Optimization Steps Taken                                                                                                                                                                                                                                                                                                |
| <ul style="list-style-type: none"> <li>None yet; caching loaded models across Streamlit reruns (e.g., via <code>st.cache_resource</code>) is a planned optimization once real models are fully in place.</li> </ul>                                                                                                     |

#### Step 5: Screenshots / Evidence







